МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОЛАТОРНОЇ РОБОТИ №2

з дисципліни **"Функційне Програмування"**

на тему **"Функції вищого порядку, lambda, композиція"**

Варіант 1

Виконала:
Студентка 2 курсу
групи ФЕП-23
Бойчук Марія

Перевірив:
Професор Свелеба С.

Львів-2026

Мета: map / filter / reduce, функціональна композиція.

Хід роботи

1. Реалізувала функції для нормалізації імен, відбору повнолітніх клієнтів та обчислення суми їхніх покупок:

```python
def normalize_names(records: list[Record]) -> list[Record]:
    return [{**r, "name": r["name"].strip().title()} for r in records]

def only_adults(records: list[Record]) -> list[Record]:
    return [r for r in records if r["age"] >= 18]

def with_total(records: list[Record]) -> list[Record]:
    return [{**r, "total": sum(r["purchases"])} for r in records]

def boost_city(records: list[Record], city: str, factor: float) -> list[Record]:
    return [
        {**r, "total": r["total"] * factor if r["city"] == city else r["total"]}
        for r in records
    ]
```

2. Створила конвеєр для обробки даних, сортування клієнтів і вибору найкращих результатів. Для обчислення статистики використала reduce:

```python
def compose(*functions):
    def result(data):
        for function in reversed(functions):
            data = function(data)
        return data

    return result
```

```python
    def sort_records(records: list[Record]) -> list[Record]:
        return sorted(records, key=lambda r: (-r["total"], r["id"]))

    def take(records: list[Record], n: int) -> list[Record]:
        return records[:n]

    def build_pipeline(
        top_n: int = 2, city: str = "Delhi", factor: float = 1.1
    ) -> Callable[[list[Record]], list[Record]]:

        def boost(records):
            return boost_city(records, city, factor)

        return compose(
            lambda records: take(records, top_n),
            sort_records,
            boost,
            with_total,
            only_adults,
            normalize_names,
        )

```

```python
def reduce_stats(records: list[Record]) -> dict[str, float]:
    result = reduce(
        lambda acc, r: {
            "count": acc["count"] + 1,
            "sum_total": acc["sum_total"] + r["total"],
        },
        records,
        {"count": 0.0, "sum_total": 0.0},
    )

    average = result["sum_total"] / result["count"] if result["count"] else 0.0

    return {
        "count": result["count"],
        "sum_total": result["sum_total"],
        "avg_total": average,
    }

```

3. Створила набір даних клієнтів інтернет-магазину та запустила програму для їх обробки:

app.py > ...

```python
from core import Record, build_pipeline, reduce_stats

records: list[Record] = [
    {"id": 1, "name": "anna", "age": 20, "city": "Delhi", "purchases": [100, 200, 150]},
    {"id": 2, "name": "ivan", "age": 17, "city": "Lviv", "purchases": [300, 100]},
    {"id": 3, "name": "olena", "age": 25, "city": "Lviv", "purchases": [400, 200]},
    {"id": 4, "name": "maksym", "age": 30, "city": "Delhi", "purchases": [500, 300]},
]

processor = build_pipeline(top_n=2, city="Delhi", factor=1.1)

result = processor(records)

print("Top clients:")
for record in result:
    print(record)

print("\nStatistics:")
print(reduce_stats(result))
```

4. Програма вивела двох клієнтів із найбільшою сумою покупок та підрахувала загальну статистику:

```
PS C:\Users\Toshiba\OneDrive\Documenti\studyy\1 cem\functional_programming\lab2> py app.py
Top clients:
{'id': 4, 'name': 'Maksym', 'age': 30, 'city': 'Delhi', 'purchases': [500, 300], 'total': 880.0000000000001}
{'id': 3, 'name': 'Olena', 'age': 25, 'city': 'Lviv', 'purchases': [400, 200], 'total': 600}

Statistics:
{'count': 2.0, 'sum_total': 1480.0, 'avg_total': 740.0}
```

5. Створила тести для перевірки сортування, відсутності змін початкових даних, обчислення статистики та композиції функцій:

tests > test_core.py > ...

```python
from copy import deepcopy
from core import build_pipeline, compose, reduce_stats

def sample_records():
    return [
        {"id": 1, "name": "anna", "age": 20, "city": "Delhi", "purchases": [100, 200]},
        {"id": 2, "name": "ivan", "age": 17, "city": "Lviv", "purchases": [500]},
        {"id": 3, "name": "olena", "age": 25, "city": "Lviv", "purchases": [400, 200]},
    ]

def test_top_n():
    result = build_pipeline(top_n=1)(sample_records())
    assert len(result) == 1
    assert result[0]["id"] == 3

def test_no_mutation():
    records = sample_records()
    original = deepcopy(records)

    build_pipeline()(records)

    assert records == original

def test_reduce_stats():
    records = build_pipeline()(sample_records())
    stats = reduce_stats(records)

    assert stats["count"] == 2
    assert stats["sum_total"] == 930.0
    assert stats["avg_total"] == 465.0

def test_compose():
    function = compose(lambda x: x + 2, lambda x: x * 2)

    assert function(3) == 8
```

6. Запустила автоматичне тестування. Усі чотири тести пройшли успішно:

```
PS C:\Users\Toshiba\OneDrive\Documenti\studyy\1 cem\functional_programming\lab2> py -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.2, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Toshiba\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Toshiba\OneDrive\Documenti\studyy\1 cem\functional_programming\lab2
plugins: platformdirs-4.13.0
collected 4 items

tests/test_core.py::test_top_n PASSED                                    [ 25%]
tests/test_core.py::test_no_mutation PASSED                              [ 50%]
tests/test_core.py::test_reduce_stats PASSED                             [ 75%]
tests/test_core.py::test_compose PASSED                                  [100%]

============================== 4 passed in 0.19s ==============================
```

7. Перевірила форматування коду за допомогою Black:

```
PS C:\Users\Toshiba\OneDrive\Documenti\studyy\1 cem\functional_programming\lab2> py -m black .
reformatted C:\Users\Toshiba\OneDrive\Documenti\studyy\1 cem\functional_programming\lab2\tests\test_core.py
reformatted C:\Users\Toshiba\OneDrive\Documenti\studyy\1 cem\functional_programming\lab2\core.py

All done! ✨ 🍰 ✨
2 files reformatted, 1 file left unchanged.
```

8. Перевірила код за допомогою Ruff та виправила зауваження:

```
PS C:\Users\Toshiba\OneDrive\Documenti\studyy\1 cem\functional_programming\lab2> py -m ruff check .
All checks passed!
```

9. Перевірила правильність типізації коду за допомогою Муру:

```
PS C:\Users\Toshiba\OneDrive\Documenti\studyy\1 cem\functional_programming\lab2> py -m mypy core.py app.py
Success: no issues found in 2 source files
```

Висновок: Під час лабораторної роботи я навчилася використовувати функції вищого порядку, lambda та композицію функцій. Реалізувала обробку даних клієнтів інтернет-магазину та перевірила програму за допомогою тестів.